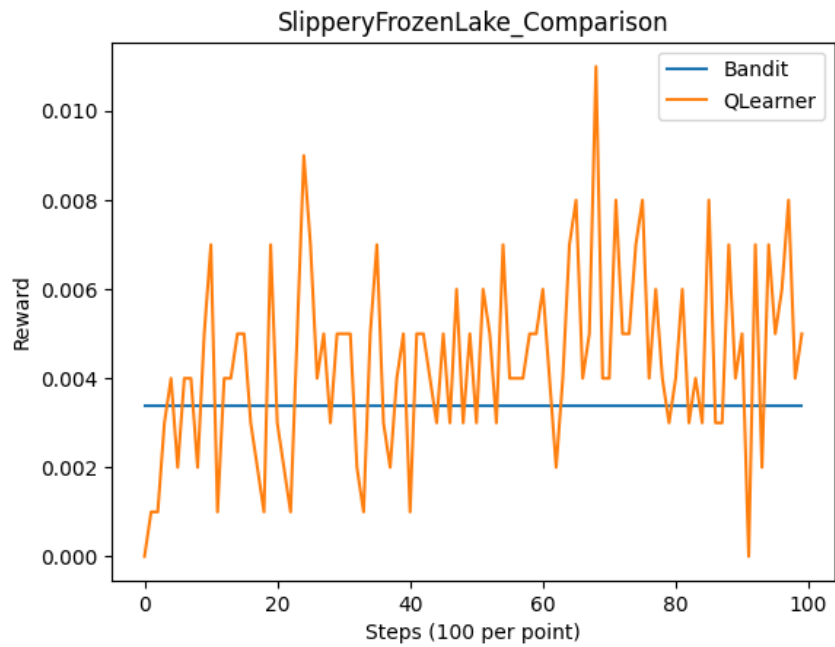
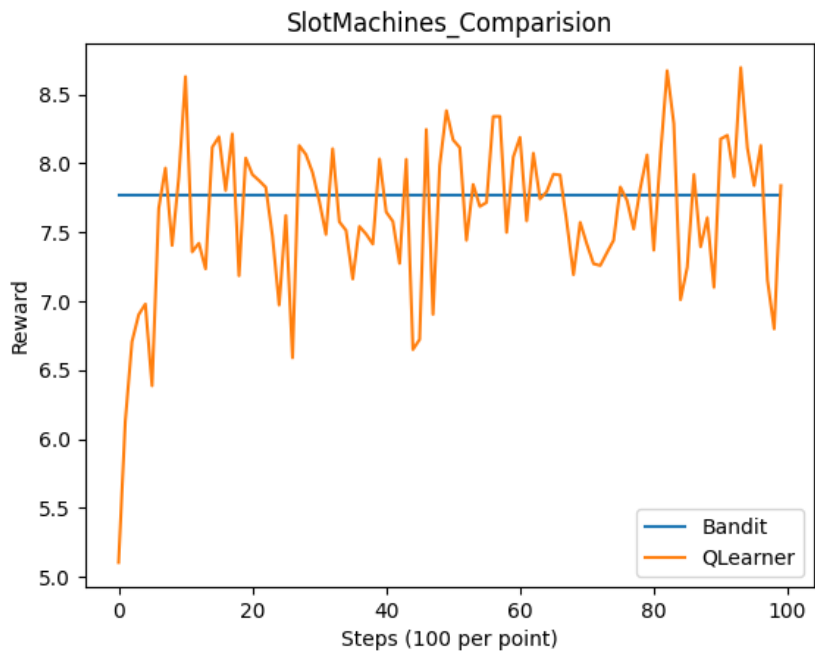
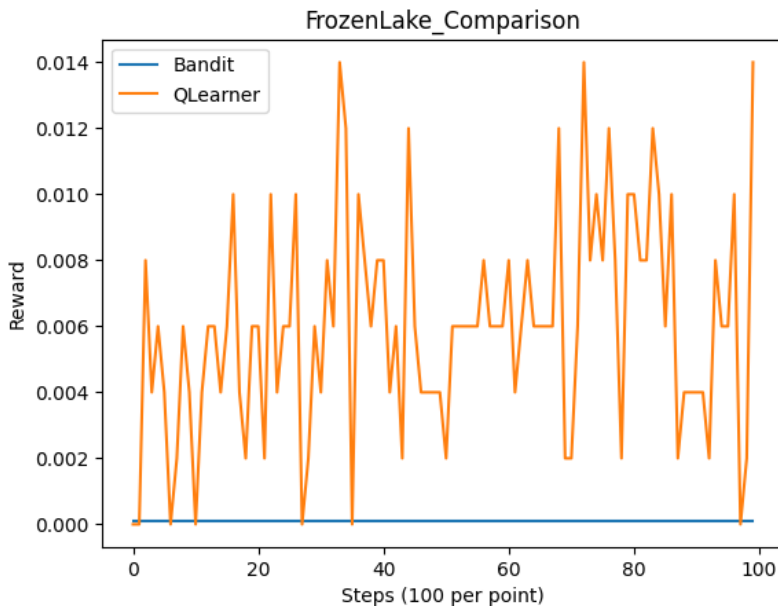




For the graph above, Bandit seems to have a lower average reward than QLearner.



QLearner seems to oscillate around Bandit therefore we can conclude that the average reward of both Bandit and QLearner could be the same



QLearner has a higher reward while Bandit has a reward of 0.

b. In which of the above plots does QLearning appear to receive higher rewards on average than MultiArmedBandit? Provide an explanation for why that happens, based on your understanding of QLearning.

From the graphs, it appears that QLearning receives higher rewards on average than MultiArmedBandit in graph 1 and graph 3.

In QLearning, the agent learns an action-value function that estimates the expected reward for each action in a given state. This function is updated using a learning rate and a reward signal received from the environment. On the other hand, MultiArmedBandit is a simpler reinforcement learning algorithm where the agent selects an action at random and receives a reward.

In graph 1, QLearning appears to have a higher average reward than MultiArmedBandit. This is likely because QLearning is able to learn which actions are more likely to result in higher rewards over time, while MultiArmedBandit is selecting actions at random.

In graph 3, QLearning again has a higher reward than MultiArmedBandit, with MultiArmedBandit receiving a reward of 0. This could be because QLearning is able to learn from past experiences and adjust its action selection accordingly, while MultiArmedBandit continues to select actions at random without any learning.

In graph 2, it is not clear which algorithm has a higher average reward as they appear to oscillate around each other. It is possible that both algorithms are receiving similar rewards on average in this case.

c. Following b.: in the environment(s) where MultiArmedBandit was the *worse*** model, is there any way you could change your choice of hyperparameters so that MultiArmedBandit would perform as well as QLearning? Why or why not?**

If MultiArmedBandit was the worse model in a given environment, it is unlikely that changing the hyperparameters alone would make it perform as well as QLearning. This is because MultiArmedBandit is a simpler algorithm that does not take into account past experiences or adjust its action selection based on those experiences, whereas QLearning does.

Hyperparameters can affect the performance of an algorithm, but they are not a substitute for the inherent limitations of the algorithm itself. In the case of MultiArmedBandit, its limitation is that it selects actions at random without any learning, while QLearning learns from past experiences to adjust its action selection.

However, if the performance of MultiArmedBandit is not too far off from QLearning, it is possible that adjusting the hyperparameters could lead to some improvement. For example, increasing the exploration rate in MultiArmedBandit could lead to it selecting more diverse actions and potentially discovering better ones. But ultimately, it is unlikely that hyperparameter tuning alone would make MultiArmedBandit perform as well as QLearning in an environment where QLearning has a clear advantage.

d. In which of the above plots does MultiArmedBandit appear to receive higher rewards on average than QLearning? Provide an explanation for why that happens, based on your understanding of MultiArmedBandit.

None of the above plots show MultiArmedBandit receiving higher rewards on average than QLearning. In fact, in all three plots, QLearning either has a higher average reward or a similar average reward to MultiArmedBandit.

MultiArmedBandit is a simple algorithm that randomly selects actions without taking into account past experiences. Its objective is to maximize the expected reward by exploring different actions and exploiting the ones that appear to have the highest reward. However, this approach does not take into account the knowledge gained from past experiences, which can lead to suboptimal decisions.

In contrast, QLearning is a more sophisticated algorithm that takes into account past experiences to learn the optimal policy. QLearning uses an iterative process to update its estimate of the optimal action-value function, which is the expected value of taking a particular action in a given state and following the optimal policy thereafter. This allows QLearning to learn from past experiences and select actions that lead to higher rewards.

Therefore, in environments where the optimal policy is complex and cannot be determined by random exploration alone, QLearning is likely to perform better than MultiArmedBandit.

e. Following d.: in the environment(s) where QLearning was the *worse** model, is there any way you could change your choice of hyperparameters so that QLearning would perform as well as MultiArmedBandit? Why or why not?**

In the environments where QLearning was the worst model, changing hyperparameters alone may not be enough to make QLearning perform as well as MultiArmedBandit.

QLearning is a model-free reinforcement learning algorithm that learns the optimal policy by iteratively updating the action-value function based on the reward obtained from the environment. The choice of hyperparameters, such as the learning rate and discount factor, can impact the convergence speed and stability of the algorithm.

However, QLearning also depends on the quality and quantity of the data it receives from the environment. In some environments, the state and action spaces may be too large or complex for QLearning to learn an accurate action-value function within a reasonable amount of time. In such cases, QLearning may require more sophisticated algorithms such as deep reinforcement learning, which can handle high-dimensional state and action spaces.

Therefore, while changing hyperparameters can improve QLearning's performance to some extent, it may not be sufficient to make it perform as well as MultiArmedBandit in all environments. A careful selection of the appropriate reinforcement learning algorithm based on the characteristics of the environment is necessary for achieving optimal performance.

